Pandas vs Polars vs Pandas 2.0 … ROUND 2

levelup.gitconnected.com/pandas-vs-polars-vs-pandas-2-0-round-2-e1b9acc0f52f

Luís Oliveira April 10, 2023

An article developed after some ideas from the first article

Source

In my previous article, I analyzed the performance between three Python libraries - Pandas 1.5.3, Polars (blazingly fast dataFrames library implemented in Rust), and Pandas 2.0 (unstable version)— in an Extract, Transform, and Load (ETL) process. This analysis used the time each library would take in each part of the process.
If you want to check my previous article please click on the link below

Pandas vs Polars vs Pandas 2.0 …. FIGHT

Testing an ETL process

levelup.gitconnected.com

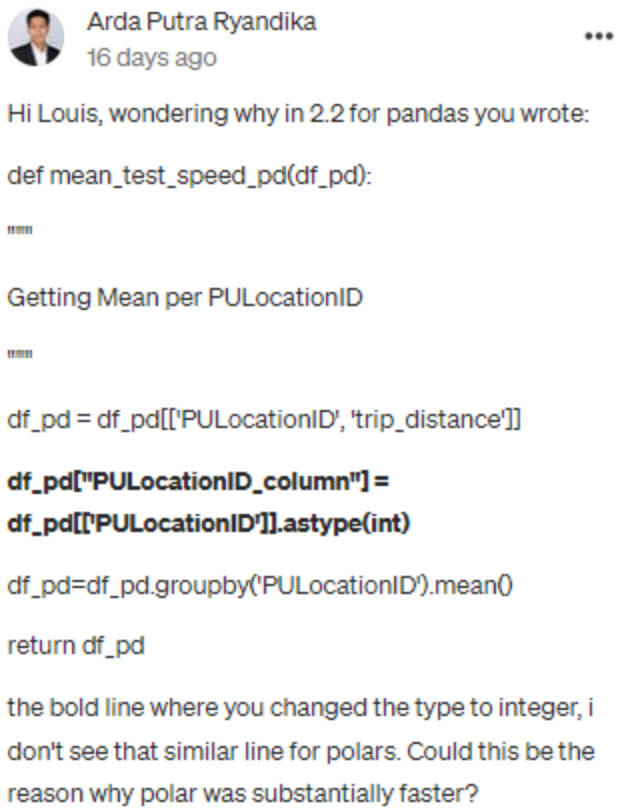
In my previous article, I had two comments that made me think that I could improve the performance analysis.

In this article I will present:

- Which were the comments and the idea I come up with from them;
- The new performance comparison.

1. Comments and Ideas

1.1. Comment #1 — Are we really comparing the same?



Arda Putra Ryandika
16 days ago

Hi Louis, wondering why in 2.2 for pandas you wrote:

```
def mean_test_speed_pd(df_pd):
"""
Getting Mean per PULocationID
"""
df_pd = df_pd[['PULocationID', 'trip_distance']]
df_pd["PULocationID_column"] = df_pd[['PULocationID']].astype(int)
df_pd=df_pd.groupby('PULocationID').mean()
return df_pd
```

the bold line where you changed the type to integer, i don't see that similar line for polars. Could this be the reason why polar was substantially faster?

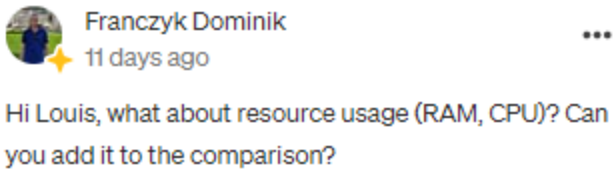
Arda Putra Ryandika has a point here (I was doing an extra step)! 😉 In Polars when we perform a *read_csv* and a *read_parquet* we get the same datatype for all the columns while in Pandas 1.5.3 we get a Numpy datatype for *read_csv* and we get a Pyarrow datatype for *read_parquet*.

While in Polars we don't have any problem with joining columns in Pandas 1.5.3 we have to convert (sometimes) one column to a certain datatype.

I must admit that I forgot to change this problem for Pandas 2.0 😏 so what I decided to do to get a better comparison was:

- Create a parquet file of the trip zones;
- Always using the read_parquet for the 3 libraries;
- Remove the read_csv function from my analysis.

1.2. Comment #2 — And RAM or CPU?



In this comment, Franczyk Dominik challenged me to compare RAM and CPU usage.

I didn't know exactly how to do that so I had to do some research on this and found out the library psutilfor comparing RAM and CPU usage and decided to apply the modules of this library as decorators.

But first of all: What are decorators?
"A decorator is a design pattern in Python that allows a user to add new functionality to an existing object without modifying its structure. Decorators are usually called before the definition of a function you want to decorate" (source https://www.datacamp.com/tutorial/decorators-python)

After understanding the concept of Python decorators I saw the article below which was great for my performance analysis.

This article refers that "all you have to do is add the decorator function and the process_memory function in your code and this will give you the memory consumed by the code and it's before and after.".

Monitoring memory usage of a running Python program - GeeksforGeeks

Managing memory is important in any programming logic but this becomes necessary for python. As python is used in Ml…

www.geeksforgeeks.org

So I tried to calculate 1) the RAM before, after, and on each function, and 2) the percentage of CPU currently in use.

2. Code

2.1. Changes in the extraction, and transformation

In my files, I decided to join all extraction functions in a single function and did the same approach on the transformation part.

Besides, according to the previous section, I removed the read_csv function and no transformation is necessary between datatypes.

The code for **Extraction** and **Transformation** for Pandas and Polars is presented below

For extraction:

```
defdef extraction():
    path1="yellow_tripdata_2021-01.parquet"
    df_trips= pd_read_parquet(path1,)
    path2 = "taxi+_zone_lookup.parquet"
    df_zone = pd_read_parquet(path2,)

      df_trips, df_zone


defextraction():
    path1="yellow_tripdata_2021-01.parquet"
    df_trips= pl_read_parquet(path1,)
    path2 = "taxi+_zone_lookup.parquet"
    df_zone = pl_read_parquet(path2,)

          df_trips, df_zone
```

For transformation:

```
deftransformation(df_trips, df_zone):



    df_trips= mean_test_speed_pd(df_trips)

    df = pd.merge(df_trips,df_zone,how="inner", left_on="PULocationID",
right_on="LocationID",)
    df = df[["Borough","Zone","trip_distance"]]
    df = endwith_test_speed_pd(df)

     df
```

```
deftransformation(df_trips, df_zone):



    df_trips= mean_test_speed_pl(df_trips)
    df = df_trips.join(df_zone,how="inner", left_on="PULocationID",
right_on="LocationID",)
    df = df[["Borough","Zone","trip_distance"]]
    df = endwith_test_speed_pd(df)

     df
```

The full code is available here for Pandas and Polars.

2.2. Code for CPU and RAM analysis

After some analysis and tests, I decided to create a file (that you can access here) that uses:

- the function to check the memory before, after, and between each function .
- the function to check the CPU in the last 2 seconds and it is called after the function.

```
import os
import psutil

defprocess_memory():
    process = psutil.Process(os.getpid())
    mem_info = process.memory_info()
return mem_info.rss



defprocess_cpu():
"""    Getting cpu_percent in last 2 seconds    """
    cpu_usage = psutil.cpu_percent(2)
return cpu_usage



# decorator function mem
defprofile_mem(func):
defwrapper(*args, **kwargs):



        mem_before = process_memory()
        result = func(*args, **kwargs)
        mem_after = process_memory()
print("Consumed memory: {:,}".format(
            mem_before, mem_after, mem_after - mem_before))



return result
return wrapper



# decorator function cpu
defprofile_cpu(func):
defwrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        cpu_after = process_cpu()
print(f"Consumed cpu: ")

                                    result     wrapper
```

In order to run these decorators you have to call them before each function like in the image below

```
from cpu_ram import profile_mem, profile_cpu


@profile_cpu
@profile_mem
def extraction():
    path1="yellow_tripdata_2021-01.parquet"
```

3. Results

In this section, I will present to you the results (I made three runs for each test) I've got for these libraries:

- Time results after changing the script to reflect a more identical code between libraries (extract data of parquet files);
- CPU and RAM results.

3.1. Time results

```
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 1.5.3


Extracting...
Extraction Parquet end in 1.808 seconds


Transforming...
Transformation end in 0.821 seconds


Loading...
Loading end in 0.294 seconds


End ETL for Pandas version 1.5.3
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 1.5.3


Extracting...
Extraction Parquet end in 1.511 seconds


Transforming...
Transformation end in 0.081 seconds


Loading...
Loading end in 0.002 seconds


End ETL for Pandas version 1.5.3
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 1.5.3


Extracting...
Extraction Parquet end in 0.644 seconds


Transforming...
Transformation end in 0.108 seconds


Loading...
Loading end in 0.009 seconds


End ETL for Pandas version 1.5.3
```

```
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
Starting ETL for Polars


Extracting...
Extraction Parquet end in 1.167 seconds


Transforming...
Transformation end in 0.144 seconds


Loading...
Loading end in 0.029 seconds


End ETL for Polars
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
Starting ETL for Polars


Extracting...
Extraction Parquet end in 0.394 seconds


Transforming...
Transformation end in 0.042 seconds


Loading...
Loading end in 0.003 seconds


End ETL for Polars
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
Starting ETL for Polars


Extracting...
Extraction Parquet end in 0.376 seconds


Transforming...
Transformation end in 0.031 seconds


Loading...
Loading end in 0.004 seconds


End ETL for Polars
```

```
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 2.0.0rc0


Extracting...
Extraction Parquet end in 1.43 seconds


Transforming...
Transformation end in 0.568 seconds


Loading...
Loading end in 0.278 seconds


End ETL for Pandas version 2.0.0rc0
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 2.0.0rc0


Extracting...
Extraction Parquet end in 0.954 seconds


Transforming...
Transformation end in 0.149 seconds


Loading...
Loading end in 0.004 seconds


End ETL for Pandas version 2.0.0rc0
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 2.0.0rc0


Extracting...
Extraction Parquet end in 0.497 seconds


Transforming...
Transformation end in 0.105 seconds


Loading...
Loading end in 0.264 seconds


End ETL for Pandas version 2.0.0rc0
```

Regarding the new script considering the extraction of parquet files, the **final time result** was similar to the previous article with a clear winner: **Polars!** 😉

AVERAGE SECONDS	Pandas 1.5.3	Polars	Pandas 2.0.0
Extraction	1.321	0.646	0.960
Transformation	0.337	0.072	0.274
Load	0.102	0.012	0.182

3.2. CPU and RAM results

```
Starting ETL for Pandas version 1.5.3


Extracting...
Consumed memory: 92,450,816
Consumed cpu: 10.1


Transforming...
Consumed memory: 316,428,288
Consumed cpu: 24.4


Loading...
Consumed memory: 339,259,392
Consumed cpu: 35.3


End ETL for Pandas version 1.5.3
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 1.5.3


Extracting...
Consumed memory: 92,438,528
Consumed cpu: 12.7


Transforming...
Consumed memory: 315,379,712
Consumed cpu: 35.0


Loading...
Consumed memory: 340,049,920
Consumed cpu: 22.7


End ETL for Pandas version 1.5.3
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 1.5.3


Extracting...
Consumed memory: 92,217,344
Consumed cpu: 23.7


Transforming...
Consumed memory: 316,481,536
Consumed cpu: 49.7


Loading...
Consumed memory: 341,106,688
Consumed cpu: 45.7


End ETL for Pandas version 1.5.3
```

```
● @guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
  Starting ETL for Pandas version 1.5.3


  Extracting...
  Consumed memory: 85,954,560
  Consumed cpu: 33.3


  Transforming...
  Consumed memory: 309,903,360
  Consumed cpu: 36.0


  Loading...
  Consumed memory: 333,066,240
  Consumed cpu: 68.2


  End ETL for Pandas version 1.5.3
● @guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
  Starting ETL for Pandas version 1.5.3


  Extracting...
  Consumed memory: 85,008,384
  Consumed cpu: 100.0


  Transforming...
  Consumed memory: 307,269,632
  Consumed cpu: 89.9


  Loading...
  Consumed memory: 331,194,368
  Consumed cpu: 63.9


  End ETL for Pandas version 1.5.3
● @guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
  Starting ETL for Pandas version 1.5.3


  Extracting...
  Consumed memory: 94,228,480
  Consumed cpu: 33.1


  Transforming...
  Consumed memory: 319,647,744
  Consumed cpu: 26.4


  Loading...
  Consumed memory: 342,347,776
  Consumed cpu: 28.2


  End ETL for Pandas version 1.5.3
```

```
@guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
Starting ETL for Polars


Extracting...
Consumed memory: 111,771,648
Consumed cpu: 43.1


Transforming...
Consumed memory: 336,232,448
Consumed cpu: 11.3


Loading...
Consumed memory: 350,470,144
Consumed cpu: 16.0


End ETL for Polars
@guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
Starting ETL for Polars


Extracting...
Consumed memory: 111,853,568
Consumed cpu: 52.6


Transforming...
Consumed memory: 336,187,392
Consumed cpu: 35.0


Loading...
Consumed memory: 350,351,360
Consumed cpu: 37.8


End ETL for Polars
@guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
Starting ETL for Polars


Extracting...
Consumed memory: 111,009,792
Consumed cpu: 10.6


Transforming...
Consumed memory: 339,935,232
Consumed cpu: 7.1


Loading...
Consumed memory: 350,883,840
Consumed cpu: 20.9


End ETL for Polars
```

```
● @guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
  Starting ETL for Polars


  Extracting...
  Consumed memory: 112,463,872
  Consumed cpu: 19.5


  Transforming...
  Consumed memory: 339,193,856
  Consumed cpu: 4.3


  Loading...
  Consumed memory: 353,624,064
  Consumed cpu: 14.5


  End ETL for Polars
● @guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
  Starting ETL for Polars


  Extracting...
  Consumed memory: 114,327,552
  Consumed cpu: 10.6


  Transforming...
  Consumed memory: 340,541,440
  Consumed cpu: 12.9


  Loading...
  Consumed memory: 352,595,968
  Consumed cpu: 16.0


  End ETL for Polars
● @guoliveira ➜ /workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python polars_performance2.py
  Starting ETL for Polars


  Extracting...
  Consumed memory: 108,748,800
  Consumed cpu: 44.4


  Transforming...
  Consumed memory: 339,431,424
  Consumed cpu: 53.4


  Loading...
  Consumed memory: 353,562,624
  Consumed cpu: 29.5


  End ETL for Polars
```

```
Starting ETL for Pandas version 2.0.0rc0


Extracting...
Consumed memory: 92,360,704
Consumed cpu: 8.0


Transforming...
Consumed memory: 324,444,160
Consumed cpu: 14.5


Loading...
Consumed memory: 347,156,480
Consumed cpu: 15.6


End ETL for Pandas version 2.0.0rc0
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 2.0.0rc0


Extracting...
Consumed memory: 92,262,400
Consumed cpu: 6.5


Transforming...
Consumed memory: 323,190,784
Consumed cpu: 14.4


Loading...
Consumed memory: 347,762,688
Consumed cpu: 11.9


End ETL for Pandas version 2.0.0rc0
@guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
Starting ETL for Pandas version 2.0.0rc0


Extracting...
Consumed memory: 94,818,304
Consumed cpu: 19.5


Transforming...
Consumed memory: 324,460,544
Consumed cpu: 30.6


Loading...
Consumed memory: 347,103,232
Consumed cpu: 24.9


End ETL for Pandas version 2.0.0rc0
```

```
● @guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
  Starting ETL for Pandas version 2.0.0rc0


  Extracting...
  Consumed memory: 88,641,536
  Consumed cpu: 8.6


  Transforming...
  Consumed memory: 315,027,456
  Consumed cpu: 15.2


  Loading...
  Consumed memory: 338,014,208
  Consumed cpu: 16.9


  End ETL for Pandas version 2.0.0rc0
● @guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
  Starting ETL for Pandas version 2.0.0rc0


  Extracting...
  Consumed memory: 93,691,904
  Consumed cpu: 33.8


  Transforming...
  Consumed memory: 324,272,128
  Consumed cpu: 26.5


  Loading...
  Consumed memory: 346,931,200
  Consumed cpu: 43.8


  End ETL for Pandas version 2.0.0rc0
● @guoliveira ➜/workspaces/data-engineer-zoomcamp-project/Python_scripts (main) $ python pandas_performance2.py
  Starting ETL for Pandas version 2.0.0rc0


  Extracting...
  Consumed memory: 95,322,112
  Consumed cpu: 8.6


  Transforming...
  Consumed memory: 328,654,848
  Consumed cpu: 12.4


  Loading...
  Consumed memory: 353,124,352
  Consumed cpu: 60.5


  End ETL for Pandas version 2.0.0rc0
```

Regarding CPU and memory usage, I had one surprise. 😁

It looks like the improvement on Pandas 2.0 paid off. 🐼

AVERAGE CPU	Pandas 1.5.3	Polars	Pandas 2.0.0
Extraction	35.483	30.133	14.167
Transformation	43.533	20.667	18.933
Load	44.000	22.450	28.933

AVERAGE USAGE RAM (2sec)	Pandas 1.5.3	Polars	Pandas 2.0.0
Extraction	519	705	493
Transformation	545	465	487
Load	564	500	360

However, I must admit that I am not totally sure of these last results 😐 :

- If you see my print screens above you will see that the values of CPU and memory usage are very heterogeneous;
- CPU usage and Memory usage are not values to be calculated in instants but over a period of time.

I would like to ask the help of YOU, **an expert reader**, how could I improve this CPU and memory analysis?

Do you know a better library/alternative for analyzing CPU and memory?

Extra Resources

pandas 2.0 and the Arrow revolution (part I)

At the time of writing this post, we are in the process of releasing pandas 2.0. The project has a large number of…

datapythonista.me

Pandas 2.0 is Here (Coming Soon)

Python Pandas 2.0 just got released and here's what they added and improved (this is going to change a lot of things)

medium.com

This Library is 15 Times Faster Than Pandas

Introducing Polars: A Python library for manipulating data

betterprogramming.pub

Polars: Pandas DataFrame but Much Faster

Perform multithreaded, optimized pandas operations

towardsdatascience.com

Did you like this article? Follow me for more articles on Medium.

Level Up Coding

Thanks for being a part of our community! Before you go:

- 👏 Clap for the story and follow the author 👉
- 📰 View more content in the
- 💰 Free coding interview course ⇒

🚀👉